Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 4

Quality

Testing, configuration, and deployment readiness

Wednesday, October 28, 2026

Your five-day journey

Yesterday the API became private. Today we prove it and prepare it to move.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD endpoints and validation

DONE

03

Security

Tokens, ownership, isolation

DONE

04

Quality

8 tests, settings from the environment

TODAY

05

Publish

Deploy, verify, README, live demo

Today's session

Learn how to prove behavior, then make the project ready for production

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Verify API behavior with automated tests

02 Read a failing test to locate the broken behavior

03 Check both the response and the database effect

04 Move secrets and runtime settings into environment variables

05 Prepare dependencies and configuration for deployment

A test is an executable promise

Arrange known data, perform one action, and assert the result



Arrange
setUp() creates fresh users, tokens, books, and tasks

Assert both directions
Allowed work succeeds. Forbidden or invalid work fails and changes nothing.

Test isolation

Each test gets a temporary, empty database. Your db.sqlite3 is never touched.

APITestCase
temporary empty database

setUp()
runs before every test

No fixtures
each test creates its own data

Runs alone
no test relies on another

```
class ReadingListAPITests(APITestCase):
    def setUp(self):
        self.alice = User.objects.create_user(username="alice", ...)
        self.alice_token = Token.objects.create(user=self.alice)

    def test_no_token_returns_401(self):
        response = self.client.get("/api/reading-list/")
        self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)
```

What makes a test meaningful?

The assertion should explain what the API promised

Status code
did it succeed or fail correctly?

Response body
did the important field or error appear?

Database
did the write happen, or stay unchanged?

Case	What it proves
Positive	allowed work succeeds
Negative	forbidden or invalid work fails
Persistence	the database reflects the response
Isolation	no test relies on another test

Automating the security boundary

Day 3's two-user check becomes repeatable evidence

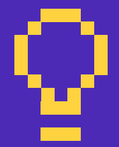
Unauthenticated
no token → expect 401

Own data
list and change → expect success

Other user's data
retrieve, update, delete → expect 404

- Create with a client-supplied owner; check the server used the token's user
- Call refresh_from_db() before checking that forbidden work changed nothing

Project: the example test plus seven REQUIRED tests.



LIVE DEMO

Break it on purpose

Write two tests, break the app, and read what fails

A 401 test
no token

A cross-user test
Bob vs Alice

Break get_queryset()
remove the filter

Read the failure
then restore the fix

A passing suite is evidence that the API still keeps its promises

Reading a failing test

Start at the bottom of the failure and work back to the behavior

AssertionError: 200 != 404

Line 41
which assertion failed?

Test name
which behavior is broken?

Conclusion

The app let another user edit a record. Fix the application, not the test expectation.

FAIL means an assertion was wrong. ERROR means the test crashed before its assertion.

Configuration belongs in the environment

The code reads runtime values instead of storing secrets in the repository

Variable	Type	Purpose
`SECRET_KEY`	string	private runtime value
`DEBUG`	boolean	local or production mode
`ALLOWED_HOSTS`	list	approved host names

```
SECRET_KEY = os.getenv("SECRET_KEY", "development-only-key")
DEBUG = os.getenv("DEBUG", "True").lower() == "true"
```

.env.example holds placeholders. The real .env stays ignored and never enters Git.

Deployment packages

The dependency file tells the production platform what to install

psycopg[binary]
PostgreSQL driver

dj-database-url
reads DATABASE_URL

gunicorn
production server

whitenoise
static files

requirements.txt → **pip install** → **pip check**

Your laptop and the deployment platform install the same declared versions.

HOUR 2 · EXERCISE

Test the Reading-List API

Four REQUIRED tests. Stretch tests are optional.

Core
4 tests, ~45 min

Stretch
4 more tests

Experiment
break it on purpose

Write tests only; the app is already built

Exercise: what you write

Tests that prove success, failure, and persistence

Core · 4 REQUIRED
login · own list · cross-user 404 · past date 400

Stretch · 4 tests
owner from token · delete protection · duplicate · PATCH

Experiment
break get_queryset() · read failures · restore the fix

- Replace each self.skipTest(...) line with the real test body
- Never hard-code ids: use self.alice_item.id, self.book_2.id
- Run python manage.py test -v 2 to see every test by name

Fix the application, not the expected result.

BREAK

Short break

After the break: test and prepare your Task Management API

HOUR 3 · PROJECT

Prove the Task API

Turn the Day 3 security guarantees into automated evidence

Commit Day 3 → **Copy checkpoint** → **P1** read the tests → **P2** 7 REQUIRED tests → **P3–P5** config + packages

git status must not list .env. Run the full suite and python -m pip check before committing.

Day 4 project increment

Goal: python manage.py test → OK (skipped=7)

TODO	File	What
P1	`tasks/tests.py`	read setUp, authenticate(), and the example test (no code)
P2	`tasks/tests.py`	7 REQUIRED tests, starting with the 2 built with the instructor
P3	`config/settings.py`	SECRET_KEY, DEBUG, ALLOWED_HOSTS from the environment
P4	`.env.example`	the three variables, placeholders only
P5	`requirements.txt`	4 deployment packages, then pip install -r

Required coverage (8 tests):

unauthenticated **token login** **create + owner**

own-only list **cross-user** **update** **delete** **validation**

Finish P3–P5 before any STRETCH test.

Day 4 exit check

Ready for Day 5 when all six pass

01	At least 8 meaningful tests pass; only stretch tests skipped
02	Tests cover access, ownership, CRUD, and validation
03	Settings read secrets from the environment
04	.env.example has placeholders only; no .env in Git
05	pip check reports no broken requirements
06	The commit is pushed

Tomorrow the same code goes live

Optional homework for Day 5

About 15 minutes. The README template you got today is completed on Day 5.

01 Open your Neon and Render accounts (part of the readiness check)

02 Neon: create the project tuwaiq-django-api and find the connection string

03 Render: New → Web Service, check you can see task-management-api, then stop

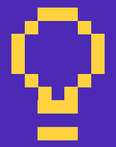
04 Read setup/deployment.md once, end to end

Do not paste the connection string anywhere yet.

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 4 complete

The API's behavior is tested and its runtime settings are ready to move.

Next: production settings, deployment, smoke test, README, and demo